

MACHINE LEARNING IN PHYSICS

GRAPH NEURAL NETWORKS

HARRISON B. PROSPER

PHY6937 / PHY4936

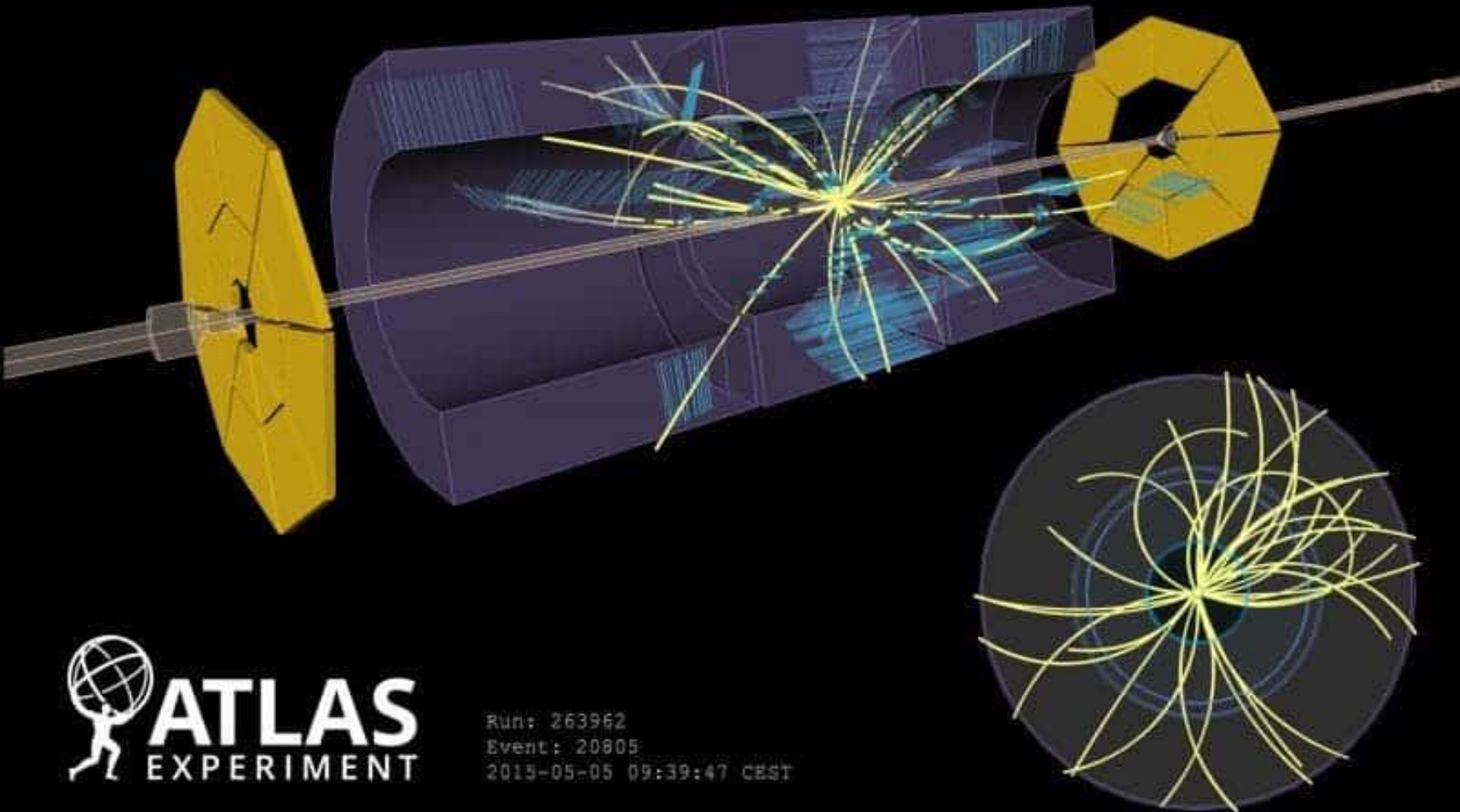
Outline

- Introduction
- Graph Neural Networks @ IceCube
- GNN Tutorial
- Summary

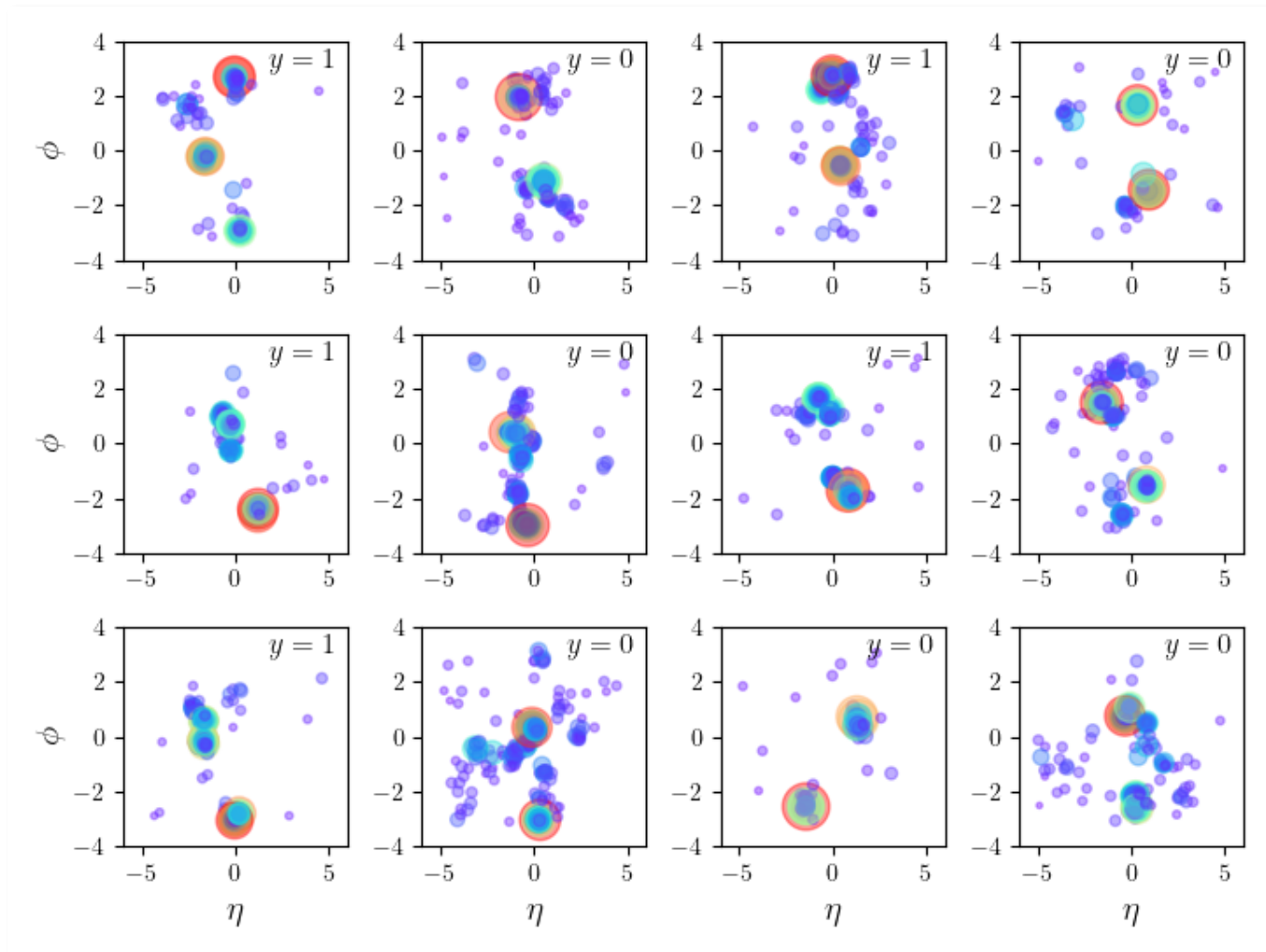
Introduction (1)

- Any sample $S = \{x_i\}_{i=1}^M$ of size M within a dataset can be regarded as a cloud of M points in a vector space, that is, as a **point cloud**.
- Standard neural networks expect the size of the point cloud S to be of fixed for every sample.
- However, there are datasets in which the size of the point cloud varies from cloud to the next. **Example:** the number of particles created in high-energy particle collision can vary from one collision event to the next.

$$\mathcal{S} = \{ (p_T, \eta, \phi)_i \}_{i=1}^M$$



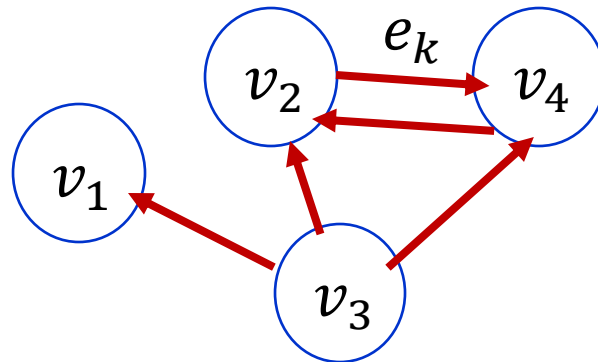
Introduction (2)



<https://lhco2020.github.io/homepage/>

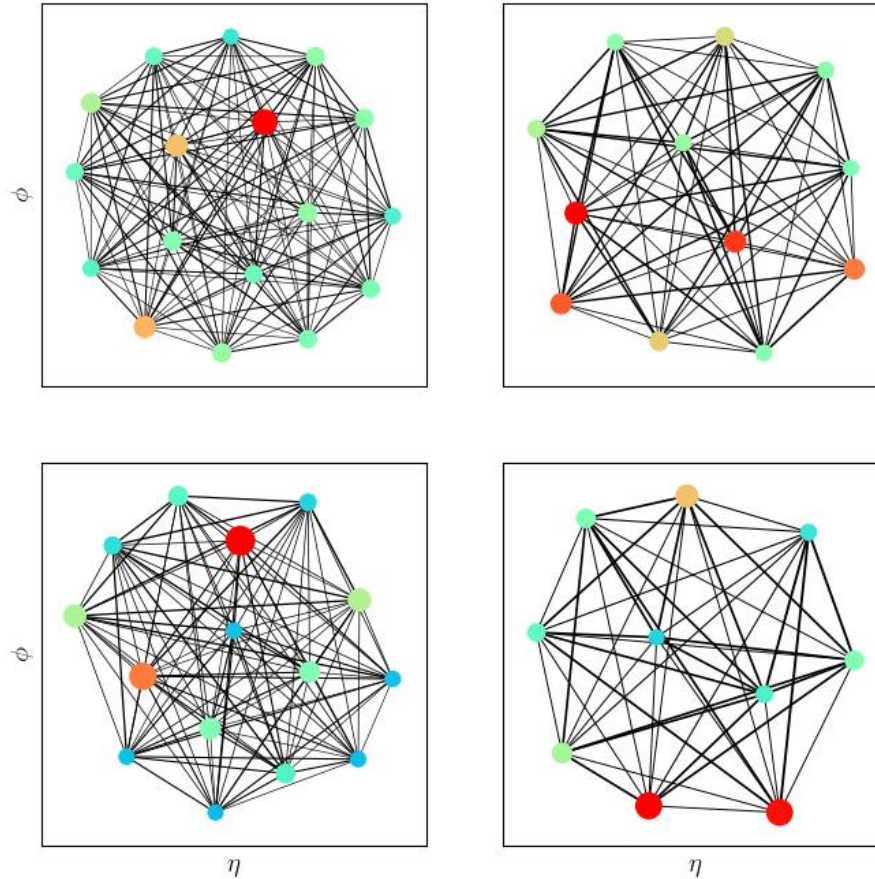
Introduction (3)

- The data from a single collision event can be modeled as a point cloud with M points in a d -dimensional space.
- One way to represent a point cloud is as a **graph**.



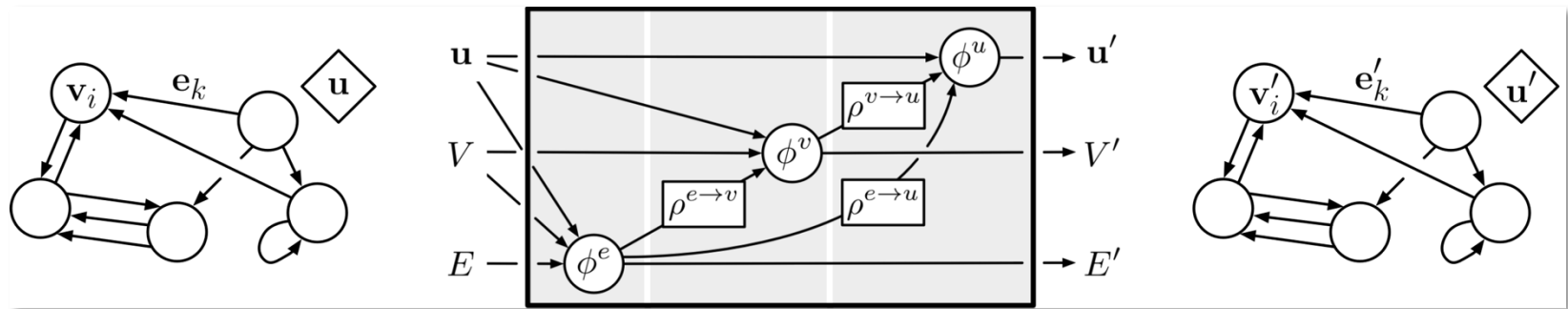
- A graph $G = (V, E)$ comprises vertices (or nodes) and edges, with attributes $V = \{v_i\}$ and $E = \{e_k\}$, respectively.

Example: Collision Events as Graphs



Introduction (4)

A graph neural network is a function that operates on data represented as graphs.

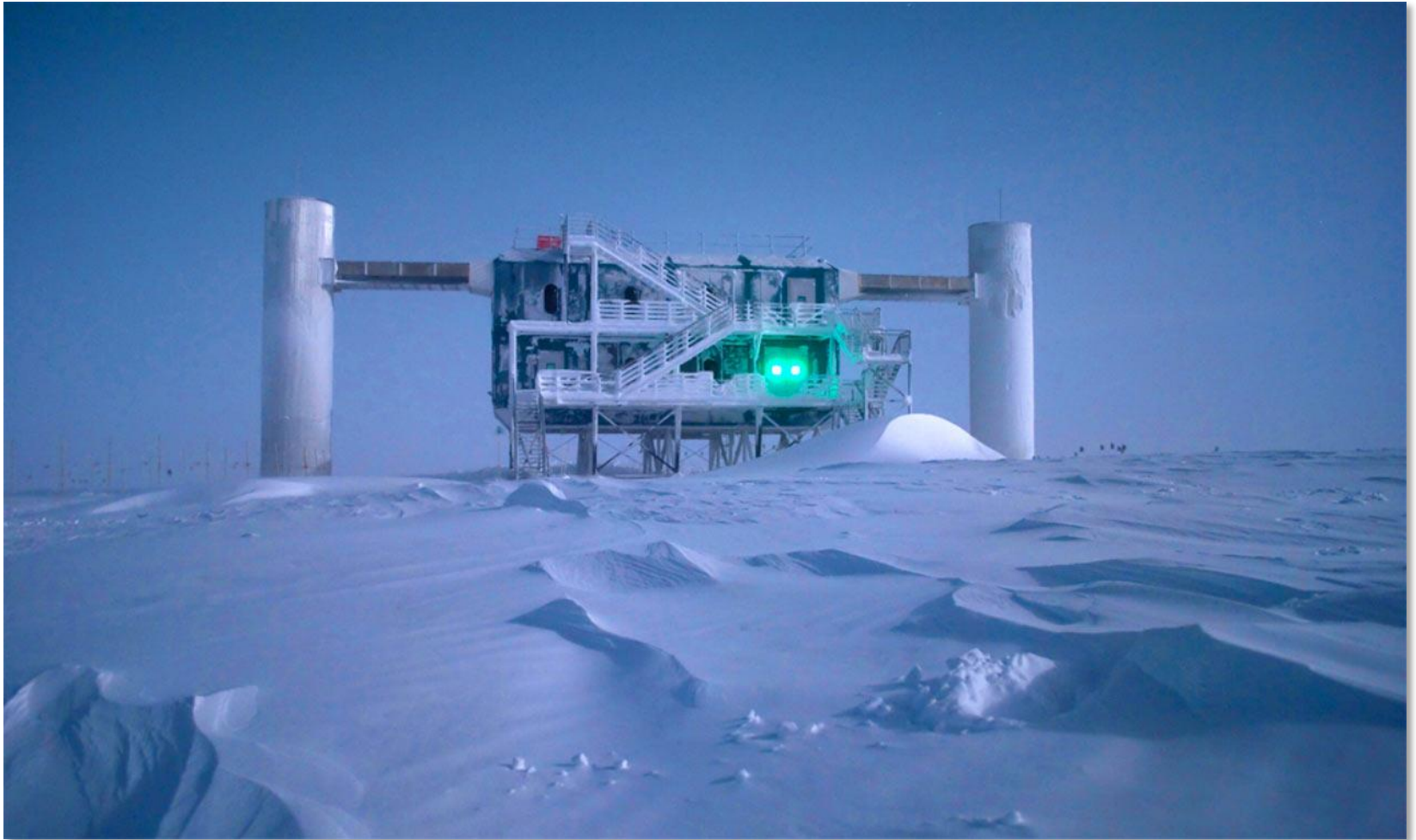


A graph enters a **graph processor** on the left and exits the processor on the right with altered graph attributes.

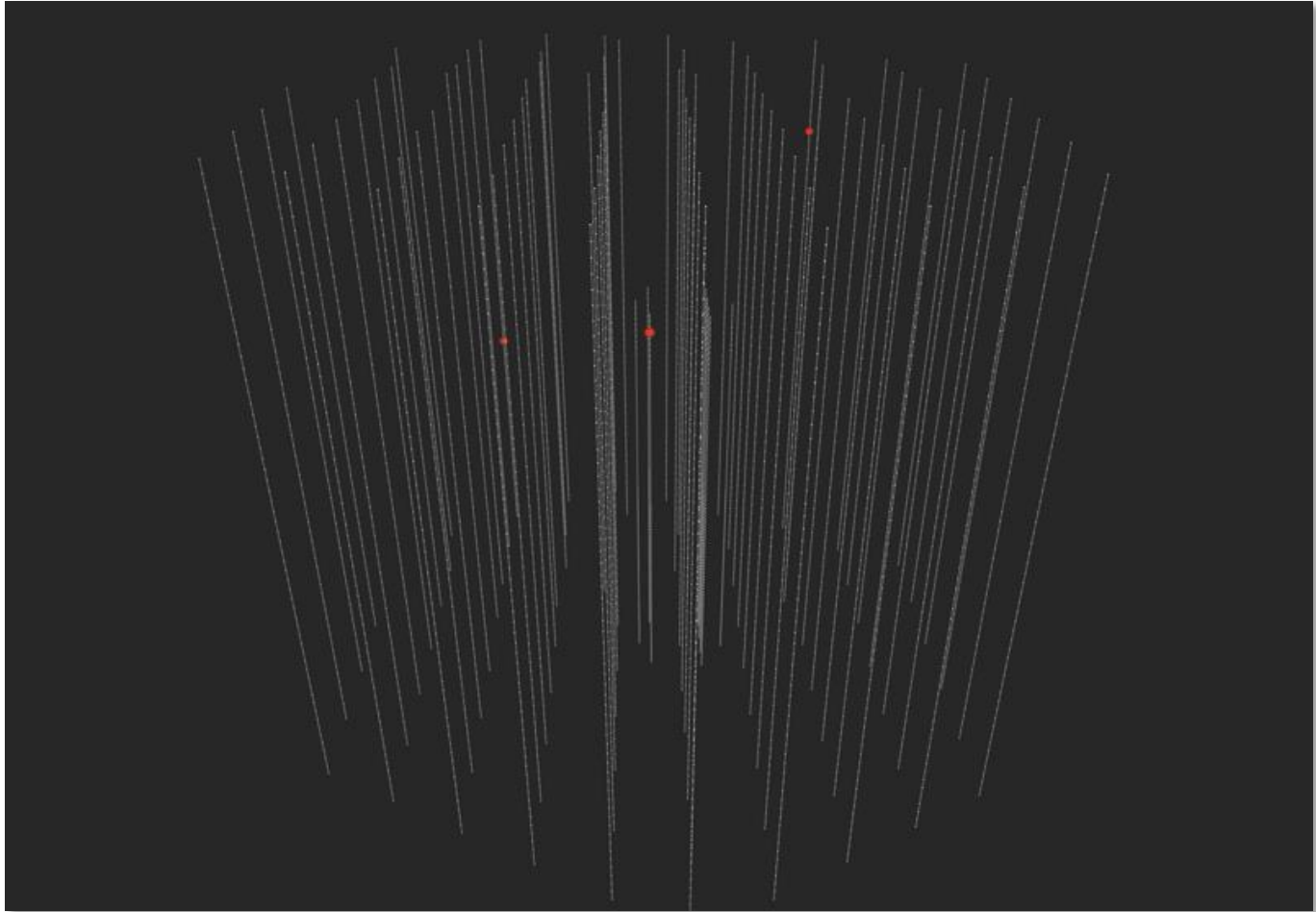
https://github.com/deepmind/graph_nets

GNNS @ IceCube

GNNS @ IceCube (1)

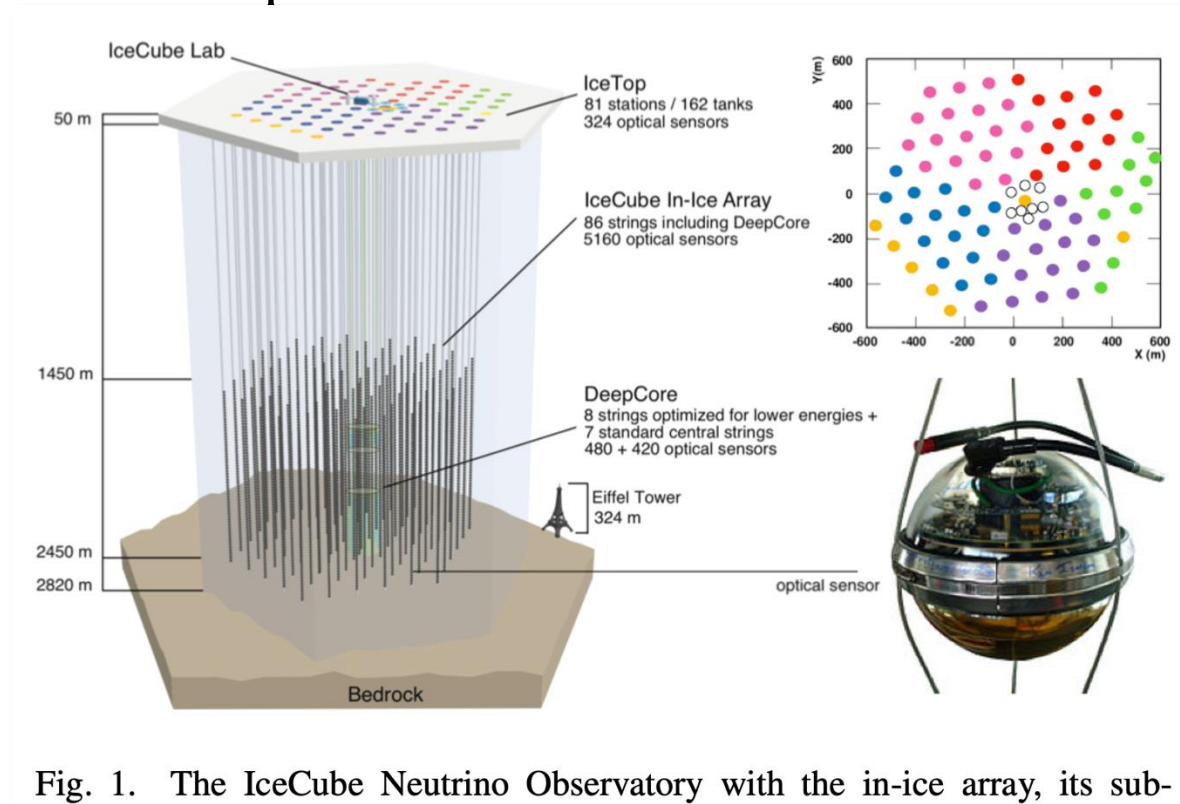


GNNS @ IceCube (2)



GNNS @ IceCube: Data (1)

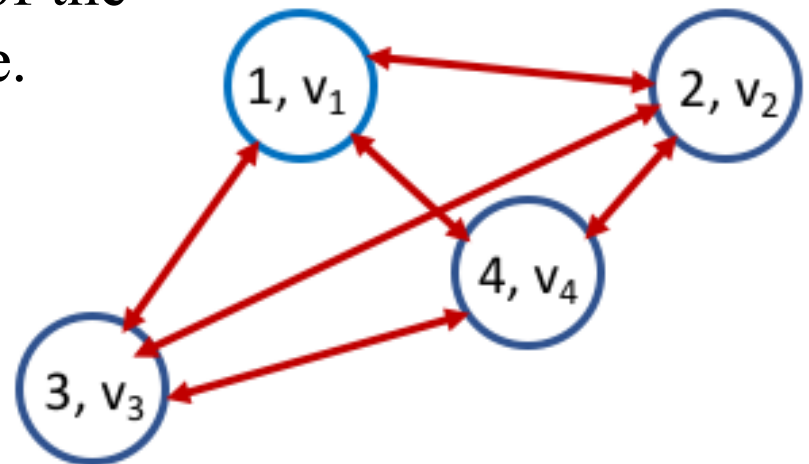
The data collected by the IceCube detector consists of signals from photomultipliers*.



*N. Choma et al. IceCube collaboration, Graph Neural Networks for IceCube Signal Classification, arXiv:1809.06166v1

GNNS @ IceCube: Data (2)

- IceCube data are the signals from n Digital Optical Modules (DOMs). The signals are modeled as the vertices of a graph.
- Each vertex is associated with a d -dimensional (row-wise) vector of attributes $\mathbf{v}_i = (x_1, \dots, x_d)_i$, three of which are the spatial coordinates (x, y, z) of the associated digital optical module.



GNNS @ IceCube: Data (3)

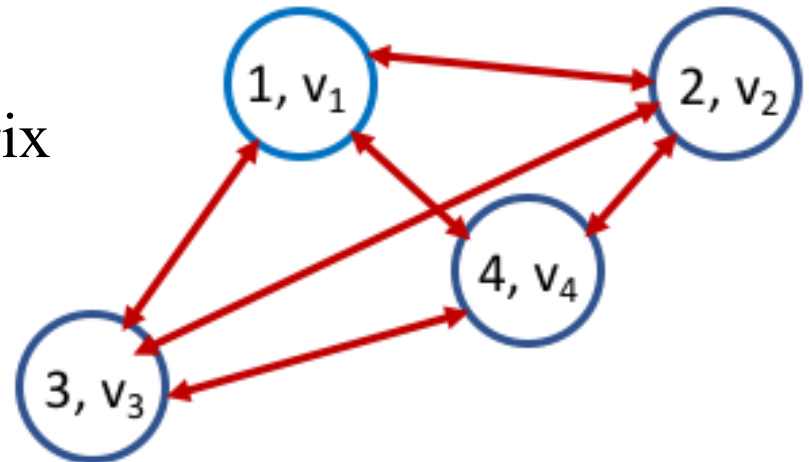
For an event with n signals, an $n \times n$ **adjacency matrix**, $A \equiv E$, is constructed using the spatial data only,

$$(A)_{ij} = \text{softmax}(d_{ij}, \text{dim} = 1),$$

where

$$d_{ij} = \exp\left(-\|x_i - x_j\|^2 / 2\sigma^2\right)$$

and σ is a free parameter. The matrix A models the edges of the graph.

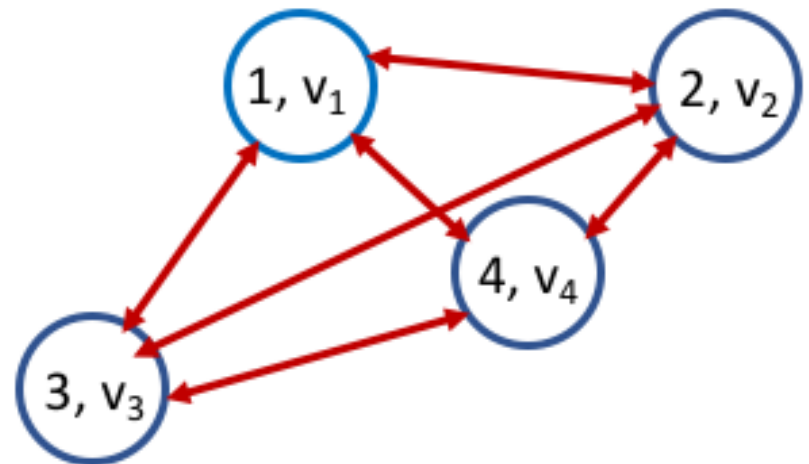


GNNS @ IceCube: Data (4)

The vectors $\mathbf{v}_i$ are concatenated vertically into an $n \times d$ matrix: $\mathbf{X} = [\mathbf{v}_1; \cdots; \mathbf{v}_n] \equiv \mathbf{V}$

$$\mathbf{X} = \begin{pmatrix} x_1 & y_1 & z_1 & \cdots \\ x_2 & y_2 & z_2 & \cdots \\ \vdots & \vdots & & \vdots \\ x_n & y_n & z_n & \cdots \end{pmatrix}$$

(Note: horizontal concatenation is denoted by $\mathbf{X} = [\mathbf{v}_1, \cdots, \mathbf{v}_n]$.)

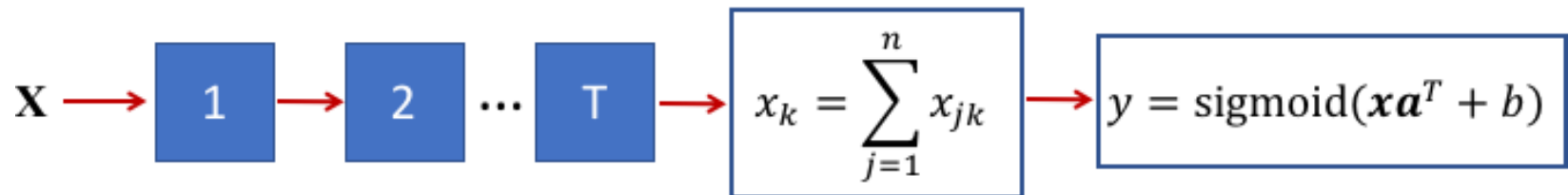


GNNS @ IceCube: GNN (1)

- The $n \times d$ matrix, $\mathbf{X}$, passes through a sequence of T graph processors, each with its own parameters.
- A graph processor is parameterized by a $2d \times d/2$ matrix of parameters, $\mathbf{w}$, and a single scalar b and computes:

$$\mathbf{Y} = [\mathbf{AX}, \mathbf{X}] \mathbf{w} + b \mathbf{I}$$

- The $\mathbf{Y}$ operation is called a **graph convolution (GConv)** and $\mathbf{I}$ is an $n \times d/2$ matrix of ones.

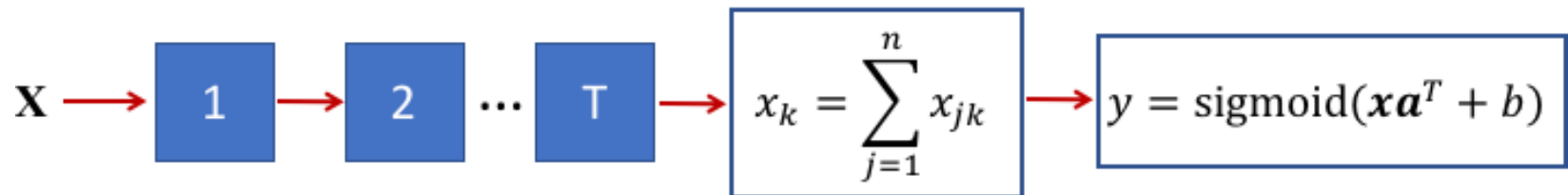


GNNS @ IceCube: GNN (2)

- In IceCube, the vertices, $\mathbf{X}$, of the output graph $G = (\mathbf{X}, \mathbf{A})$ are constructed through *horizontal* concatenation as follows:

$$\mathbf{X} = [\text{ReLU}(\mathbf{Y}), \mathbf{Y}]$$

with, as usual, the ReLU function applied *element-wise*.



GNNS @ IceCube: GConv (1)

$$\begin{aligned} Y &= [AX, X]w + bI, \\ X &= [\text{ReLU}(Y), Y] \end{aligned}$$

Implementation

AX = torch.matmul(A, X)

AXX = torch.cat([AX, X], dim=1)

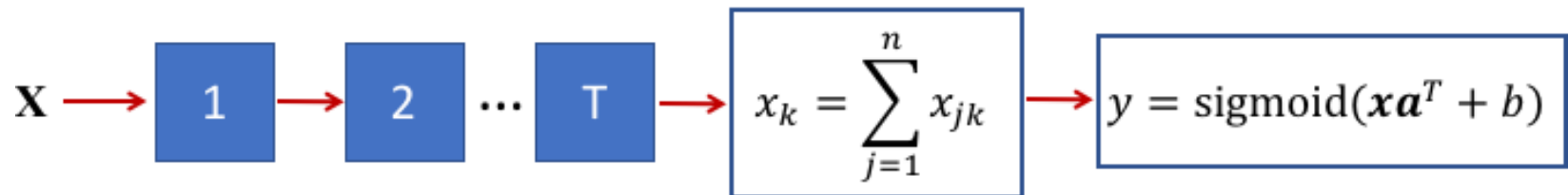
$AXXw$ = torch.matmul(AXX, w)

Y = $AXXw + b * \text{torch.ones_like}(AXXw)$

X = torch.cat([F.relu(Y), Y], dim=1)

GNNS @ IceCube: GNN (3)

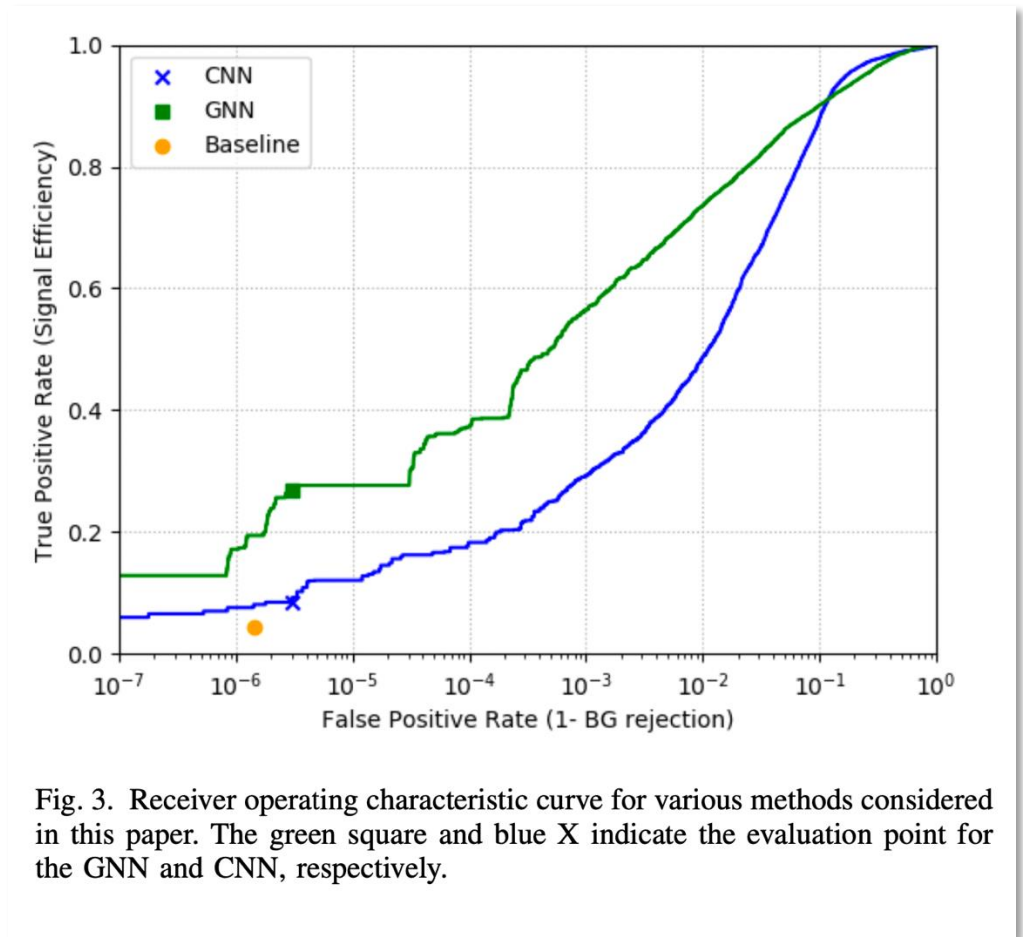
- At the end, $\mathbf{X}$ is mapped to a d -dimensional vector $\mathbf{x}$ using a *permutation invariant map* with respect to the vertices and *invariant* with respect to the number of vertices.
- Finally, the vector $\mathbf{x}$ is mapped to the scalar output $0 \leq y \leq 1$ using a *sigmoid*.



GNNS @ IceCube: GNN Results

The GNN is **6.3** times more efficient than the IceCube physics baseline analysis at a signal to noise ratio that is **3** times better.

It also outperforms a 3D CNN.



GNN TUTORIAL

```
class GNNConv(nn.Module):
```

```
    def __init__(self, in_features, out_features, scale=0.05):
```

```
        # initialize base class
```

```
        super().__init__()
```

```
        # embed the "n" input vertices in a larger vector space
```

```
        # of dimension h = out_features
```

```
        self.embed = nn.Linear(in_features, out_features)
```

```
        self.weight = nn.Parameter(scale *  
                                    (2*torch.rand(out_features, out_features) - 1.0) )
```

```
        self.bias = nn.Parameter(scale * ( 2*torch.rand(1) - 1.0) )
```

```
def forward(self, X, A):
```

```
    # embed vertices in a larger vector space
```

```
    X = self.embed(X)
```

```
    X = F.relu(X)
```

```
    # perform graph convolution: for each vertex, compute a
```

```
    # weighted sum of the features of all other vertices,
```

```
    # weighting the nearest neighbors more heavily.
```

```
    #  $AX = A * X$ 
```

```
    AX = torch.matmul(A, X)
```

```
    # compute  $(A * X) * W + B * I$ 
```

```
    X = torch.matmul(AX, self.weight) + self.bias
```

```
    X = F.relu(X)
```

```
    # return modified vertices
```

```
    return X
```

```
class InvariantFunction(nn.Module):  
    """  
    Perform a permutation and vertex count-invariant operation on the  
    vertices of a graph.  
    """  
    def __init__(self):  
        super().__init__()  
  
    def forward(self, x):  
        m = x.mean(dim=1)  
        return m
```



```
class GNN(mlp.Model):

    def __init__(self, n_dim=3, n_embed=40, dropout=0.5):

        # initialize base class
        super().__init__()

        self.edges    = Edges()
        self.gconv1    = GNNConv(n_dim,  n_embed)
        self.gconv2    = GNNConv(n_embed, n_embed)
        self.invarfun   = InvariantFunction()
        self.linear     = nn.Linear(n_embed, 1)
        self.dropout   = nn.Dropout(dropout)
```

```
def forward(self, V):  
    # construct n x n adjacency matrix which together  
    # with the vertices V forms the graph  $G = (V, E)$   
    E = self.edges(V)  
  
    # pass graph  $G = (V, E)$  through multiple  
    # graph convolution layers (graph processors)  
    V = self.gconv1(V, E); V = F.relu(V); V = self.dropout(V)  
    V = self.gconv2(V, E); V = F.relu(V); V = self.dropout(V)  
  
    # apply a permutation and vertex count-invariant  
    # function to the vertices  
    x = self.invfun(V)  
  
    # complete with a simple MLP  
    x = self.linear(x); x = F.sigmoid(x)  
  
    return x
```

Summary

- Data instances in which the number of points, or vertices, can vary can be modeled with graphs $G = (V, E)$ using graph neural networks (GNN).
- We looked at a specific application of GNNs by IceCube, which, for this experiment, achieved state-of-the-art classification results.